A Novel Application of Ant Colony Optimization to k-defective Maximum Edge Bicliques

Kiyaan Pillai

September 2026

Abstract

The maximum k-defective edge biclique (k-MDB) problem is an emerging NP-hard optimization. We adapt Ant Colony Optimization (ACO) to this problem and introduce a smaller side preference to outperform previously proposed heuristics. In some instances, ACO produces bicliques with 3-4x more edges than previously proposed heuristics. Additionally, ACO provides a tighter upper bound on the size of the k-MDB and can reduce brute force search time by up to 95% on net end-to-end time (including ACO search cost), though compute constraints limit the robustness of our test. This work examines a new heuristic approach to the k-MDB problem while identifying several improvements for future heuristics.

1 Introduction

A bipartite graph is a common graph structure where one set of vertices is connected solely to another. Bipartite graphs can model user-product behavior, person-place interaction, and more. A biclique is a set of vertices on a bipartite graph such that each vertex on one side is connected to each vertex on the other. The maximum edge biclique problem aims to identify the biclique with the most edges on a bipartite graph. The problem has applications in online recommendation systems, fraud detection, and community detection [1].

Consider an e-commerce platform that models buyers and products as a bipartite graph. A coordinated fraud ring may produce a block of accounts that purchase overlapping sets of items, yielding a subgraph that is almost fully connected across the two sides; requiring a perfect biclique would discard this signal whenever a few transactions are omitted deliberately or lost to incomplete logging. A similar pattern arises in recommendation, where a large set of users who rated nearly the same catalog of movies still indicates a meaningful community worth surfacing, even when a handful of ratings are missing. In both settings, what matters is not perfect completeness but a dense, near-bipartite structure that remains interpretable under small amounts of noise.

More generally, real-world data often introduces noise to graphs, and structures with relatively few missing connections may still be relevant. A biclique that is allowed up to k missing edges, or k-defective biclique (k-DB), can overcome this real-world noise. Identifying the maximum k-defective edge biclique (k-MDB) remains an emerging field.

The k-MDB problem is NP-hard. Unless P = NP, there is no algorithm that can run in polynomial time and is guaranteed to find the maximum solution. When the absolute optimum is not necessary, approximation techniques are useful to significantly reduce computation while providing meaningfully large bicliques.

Dorigo et al. [2] proposed a metaheuristic for exploring graphs known as Ant Colony Optimization (ACO). Initially used on identifying the shortest path in problems such as the Traveling Salesman Problem, ACO has been adapted to many graph problems, such as vehicle routing [5], quadratic assignment [6], graph coloring [7], and the maximum clique problem [8], the last of which is closely related to the k-MDB problem via the reduction discussed in Section 2.2. Several improvements have been proposed to the algorithm since its inception, most notably the MAX-MIN Ant System (MMAS) [4]. We adapt MMAS as a heuristic approach to identifying the k-MDB. We observe that it performs significantly faster than brute-force algorithms while still returning comparably large k-defective bicliques.

2 Background

2.1 Definitions

Define a bipartite graph as a graph $G = (U, V, E)$, where U and V are two disjoint sets of vertices, and $E \subseteq U \times V$ is the set of edges between U and V. Define $n = |U| + |V|$ and $m = |E|$. Let $S = (U_S, V_S)$ denote a subset of vertices where $U_S \subseteq U$ and $V_S \subseteq V$. Let $G(S) = (U_S, V_S, E_S)$ be the induced subgraph of S where $E_S = \{(u, v) \mid u \in U_S \wedge v \in V_S \wedge (u, v) \in E\}$. Define $\bar{E}(S) = \{(u, v) \mid u \in U_S \wedge v \in V_S \wedge (u, v) \notin E\}$ be the set of non-edges in a subgraph. Let a k-defective biclique be a subgraph S where $|\bar{E}(S)| \leq k$.

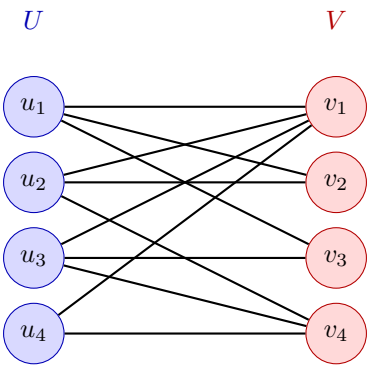

Figure 1: Example bipartite graph $G = (U, V, E)$

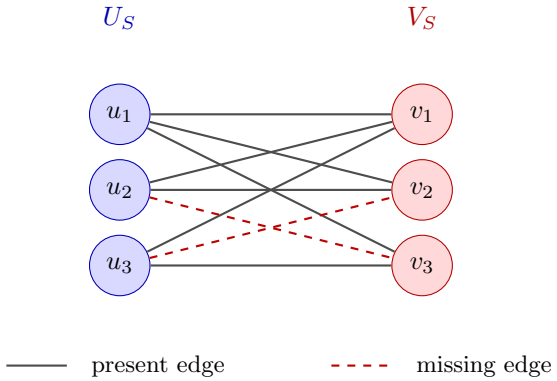

Figure 2: A 2-defective biclique present in Figure 1. Every vertex in U_S is connected to every vertex in V_S except for two missing edges. Since $|\bar{E}(S)| = 2$, this is a k-defective biclique, where $k \geq 2$.

Define the neighbors of a node u in a subgraph S as $n_S(u) = \{v \in S \mid \{u, v\} \in E(S)\}$. Define the nonneighbors of a node as $\bar{N}_S(u) = \{v \in S \mid (u, v) \notin E(S)\}$. Define the degree of a node as $d_S(u) = |n_S(u)|$, and the nondegree as $\bar{d}_S(u) = |\bar{N}_S(u)|$.

Define a maximal k-MDB as a subgraph S that cannot be expanded. That is, for any node $u \notin S$, $|\bar{E}(S \cup \{u\})| > k$. Define the maximum k-MDB as the largest maximal k-MDB in a graph G. Let θ be a user-defined limitation on the solution space. The algorithms proposed in this paper seek to find the maximum k-MDB where both sides have at least θ vertices. This is because, on real-world graphs, k-defective bicliques can be formed with many edges but disproportionate vertex counts, where one side has very few vertices while the other has many [1]. Without the θ term, the utility of the algorithm is limited.

2.2 Computational Complexity

Finding the maximum k-MDB on a bipartite graph is an NP-hard optimization problem. The problem reduces to the well-known NP-hard maximum clique problem, which seeks to find the maximum set of vertices on a graph connected to every other vertex [1].

2.3 Previous Work

Related dense subgraph models provide context for the k-MDB formulation. In unipartite graphs, *defective cliques*—subgraphs missing at most a bounded number of edges—were introduced to recover nearly complete structure from noisy biological interaction networks [12]. Subsequent exact algorithms for the maximum k-defective clique problem have primarily used branch-and-bound frameworks with graph reduction and upper-bound pruning [13], [14]. On bipartite graphs, relaxed models include k-biplexes, which permit each vertex a fixed number of cross-side non-neighbors and can proliferate as k grows [15], and k-defective bicliques, which bound missing edges globally and generalize bicliques when $k = 0$ [1], [11]. Across both settings, the dominant algorithmic toolkit combines exact search, aggressive graph reduction, and construct-and-expand heuristics to obtain strong initial bounds.

Cui et al. [1] proposed a brute-force approach to the k-MDB problem, as well as an initial heuristic designed to find a k-MDB with exactly θ nodes on one side of the graph.

Concurrently, Wang et al. [11] studied a related but distinct formulation: the maximum k-defective biclique measured by vertex count rather than edge count, with a similar minimum vertex count on both sides. They develop an exact branch-and-bound algorithm with time complexity $\mathcal{O}^*(\gamma^{n+k})$, tightened to $\mathcal{O}^*(\gamma^{\alpha\delta^2+k})$ by exploiting a diameter-three property of large defective bicliques, and propose an analogous construct-and-expand heuristic (greedy initial construction followed by one-sided greedy expansion) to seed their search. Because their objective differs from ours, we do not benchmark directly against their algorithm, but their diameter-three pruning and expansion-based seeding strategy are directly relevant to future extensions of this work (Section 6.1).

Ant Colony Optimization is a natural metaheuristic for problems where high-quality solutions are built incrementally by adding vertices under local feasibility constraints. In the maximum clique problem, ACO algorithms construct cliques greedily while using pheromone to reinforce vertices that appear in large solutions [8]. The k-MDB problem has similar structure: an ant can grow a near-biclique vertex by vertex, checking at each step whether the defect budget permits another addition. By contrast, population-based encodings such as genetic algorithms must represent membership across both partitions simultaneously; in our experiments, a GA adapted from the GCLIQUE framework [3] underperformed both ACO and the θ-heuristic, likely because crossover and mutation on such a high-dimensional bitmask struggle to preserve the global density constraint across two vertex sets. These considerations motivate our choice of MMAS [4] over alternative metaheuristics for seeding exact search on k-MDB instances.

3 Methods

3.1 ACO Algorithm

ACO traditionally has a set of ants that traverse a graph, leaving pheromone that attracts other ants to follow their path. As the ants explore the solution space, they all converge on one particular path as the quantity of pheromones on that path becomes increasingly larger.

We adapt the ACO algorithm to the k-MDB problem. A population of ants iteratively adds vertices from G to a subgraph S, preferring nodes with higher d_S and higher pheromone τ. Here, $\sigma(x) = \frac{1}{1+e^{-x}}$ denotes the standard logistic sigmoid, used to dampen the impact of a node's degree with the entire graph as the size of the subgraph grows. Ants prefer to add nodes connected to the node they most recently visited. Intuitively, this reduces the number of nodes that an ant has to consider, and increases the likelihood that early on, an ant adds nodes that are properly connected to the subgraph.

Algorithm 1 ACO

Ensure: a subset of vertices S such that $G(S)$ is a k-MDB

```
function ACO(G, n_E, n_S, N, ρ, T)
    for all u ∈ G do
        τ(u) ← 1
    e ← 0
    B ← {}
    while e < n_E do
        e ← e + 1
        S ← n_S empty subgraphs
        τ_C ← τ
        parallel for each S_i ∈ S do
            C ← {u ∈ G | k ≥ |Ē(S ∪ {u})|}
            while |C| > 0 do
                L ← the last node that was appended to S
                C_L ← d_L
                if |C ∩ C_L| > 0 and N then
                    search ← C ∩ C_L
                else
                    search ← C
                next ← linear sample of u ∈ search based on DESIRABILITY(u, S_i, τ_C)
                add next to S_i
                τ_C(u) ← τ_C(u) + T
                C ← {u ∈ G | k ≥ |Ē(S ∪ {u})|}
        S_B ← argmax_{S_i ∈ S} INSTANCEFITNESS(S_i)        ▷ S_B is guaranteed to have at least one node
        if B = ∅ or INSTANCEFITNESS(S_B) > INSTANCEFITNESS(B) then
            B ← S_B
        τ ← τ_C
        for all u ∈ G do
            τ(u) ← τ(u) × (1 − ρ)
    return B
```

We observe that ACO has trouble generating graphs with at least θ nodes because it often adds a disproportionate number of nodes on one side. We first introduce an instance fitness function that rewards a more balanced biclique by squaring the minimum number of nodes. For the desirability function, we introduce a hyperparameter P, a factor that favors nodes on the smaller side when one side of an ant's subgraph has $\geq \theta$ nodes and another side has $< \theta$. We use the following desirability.

Algorithm 2 Prefer-Smaller-Side Desirability

```
function INSTANCEFITNESS(S)
    a, b ← minmax(|U_S|, |V_S|)
    if a ≥ θ then
        return b × a^2
    else
        return a^2
function DESIRABILITY(u, S, τ)
    η ← d_S(u) + d_G(u) × σ(|U_S| + |V_S|)
    if (|U_S| ≥ θ ⊕ |V_S| ≥ θ) and u is on the smaller side then
        η ← η × P                                  ▷ prefer node on the smaller side
    return τ(u) × η^3
```

We attempted an elitist approach to lay additional pheromone on subgraphs with higher fitnesses according to a fitness function, as well as implementing local tabu search on ant subgraphs to prioritize solutions

that are more likely to grow. We implemented the MAX-MIN Ant System as well. Finally, we attempted a genetic algorithm approach adapted from Shah's GCLIQUE framework [3], but found it underperformed both ACO and the θ-heuristic, likely due to the high dimensionality of the encoding required to represent biclique membership across both vertex sets.

3.2 Time Complexity

Let $n = |U_G| + |V_G|$ and $m = |E_G|$. For each ant, it must first examine n nodes. At the next step, it will examine a maximum of $n-1$ nodes, and then $n-2$ nodes, etc. Therefore, an ant will examine at most $n + (n-1) + (n-2) \ldots = n(n+1)/2$. Additionally, at each step, an ant generates a candidate set of nodes that could still be added to its subgraph. This pass is optimized to be $\mathcal{O}(|U_C| + |V_C|)$, where C is the set of candidate nodes before it advanced. If C remains very large, the upper time complexity would therefore be $\mathcal{O}(A \times I \times (n^2 + n))$ for A ants advancing for I iterations, but this would be dominated by n^2. In a parallelized implementation with p processors, the true upper complexity bound is $\mathcal{O}(\lceil A/p \rceil \times I \times n^2)$.

In practice, however, there are two factors limiting this complexity. First, ants run until they are unable to add more nodes. Let D be the k-MDB of G. Because smaller side preference encourages subgraphs with θ nodes on each side, an ant will empirically visit a maximum of $n_D = |U_D| + |V_D|$ nodes, and therefore have a time complexity of $\mathcal{O}(\lceil A/p \rceil \times I \times n_D \times n)$. n_D is often much lower than the total number of nodes. Additionally, let L be the last node an ant visited. An ant prefers to visit only the intersection of d_L and its candidate set, so it often only visits d_L or fewer nodes. Therefore, the practical time complexity is $\mathcal{O}(\lceil A/p \rceil \times I \times n_D \times (n + \max_{v \in G} d_v)$. Because $\max_{v \in G} d_v$ is necessarily lower than n and almost always lower than $\min(|U_G|, |V_G|)$, ACO's complexity practically does not scale linearly with n after the first iteration.

3.3 Brute Force Approach

Finally, we implement the approach proposed by Cui et al. These authors observe that many k-MDB have one side with exactly θ vertices and propose a heuristic that removes nodes from one side while adding nodes to another until there are θ nodes on that side. We summarize their algorithm below.

Algorithm 3 θ-Heuristic

Ensure: a k-defective biclique of G

```
function ThetaHeuristic(G)
    U ← {}
    V ← V_G
    while |U| < θ do
        append u ∈ U_G \ U with largest d_V to U
        while |Ē(U ∪ V)| > k do
            remove v ∈ V with largest d̄_U from V
    return U ∪ V
```

They then propose a branch-and-pivot algorithm that takes advantage of this initial heuristic upper bound. The algorithm checks combinations of $S = (U_S, V_S)$ by maintaining a candidate set $C = (U_C, V_C)$ of all remaining nodes $u \in G$ s.t. $|\bar{E}(S \cup \{u\})| \leq k$ that could still be added to S. It recursively chooses a node $u \in C$ and branches on $(S' = S \cup \{u\}, C' =$ candidate set of $S')$ and $(S' = S, C' = C \setminus \{u\})$. Several optimizations are added to make this algorithm faster than a naive brute-force approach. Its time complexity is $O(m\beta_k^n)$, where β is the largest real root of the equation $x^{2k+5} - 2x^{2k+4} + x^{k+3} - 2x + 2 = 0$. See Appendix B for full pseudocode.

The paper proposes a graph reduction technique called common neighbor reduction. Common neighbor reduction prunes edges between nodes n_1 and n_2 if n_1 has fewer than $\theta - k$ common neighbors with n_2, and prunes nodes with fewer than $\theta - k$ edges.

The θ-heuristic has a time complexity of $\mathcal{O}(\theta n + m)$ [1].

3.4 Experimental Setup

Unless otherwise noted, all ACO experiments use the following hyperparameter values, selected via preliminary tuning on a small subset of graphs:

Table 1: ACO hyperparameters used in all experiments

Parameter	Value
Evaporation rate (ρ)	0.05
Pheromone deposit (T)	1
Number of epochs (n_E)	5
Smaller-side preference factor (P)	2
k	2
θ	5

ρ was tuned manually by observing convergence behavior on a small subset of graphs; T, n_E, and P were set to reasonable fixed values without a formal search. A more systematic hyperparameter study is left to future work.

When testing ACO and the θ-heuristic, a biclique with k missing edges and θ vertices on each side was injected into the graph to make graphs without a naturally occuring k-DB usable for tests. Sometimes, graphs may have bicliques larger than the injection.

We test on 36 graphs from two public collections: 33 from KONECT (`konect.cc`), a standard repository of real-world bipartite networks, and 3 from Amazon Reviews 2023 (`2023-amazon-reviews.github.io`), user-product review graphs in selected product categories.

For the pivoting test, we test on a subset of 21 graphs. On the other 15 graphs, ACO was able to beat the θ-heuristic but the branch-and-pivot approach took longer than a timeout of 2000 seconds for both the traditional θ-heuristic and ACO seeded runs, making a time comparison on our current hardware.

First, ants were varied across $\{2, 5, 10, 20, 50, 100, 200\}$. Solution quality was measured, both in terms of edge count and the percentage of ant solutions that had more than θ nodes on both sides (referred to as θ feasibility rate). Wall clock time is also shown to scale linearly with the number of ants. We aim to examine tradeoffs between ant count and results.

Second, we hypothesize that ACO can provide a larger initial lower bound on the maximum solution than the θ-heuristic and significantly reduce branch and pivot execution time.

Third, we compare ACO-PN directly against the θ-heuristic on the full benchmark suite, measuring solution quality, θ-feasibility, and wall-clock time.

Finally, we ablate two optimizations to traditional ACO. Prefer smaller side (P) weights nodes on the smaller side of an ant's subgraph higher; neighbor scope limit (N) restricts an ant to travel only to neighbors of its most recently visited node. ACO, ACO-P, ACO-N, and ACO-PN denote traditional ACO, ACO with P, ACO with N, and ACO with both flags, respectively. We hypothesize that N improves solution quality and that P decreases execution time.

All tests were run with 8 threads on a computer with Ubuntu, an Intel(R) Core(TM) Ultra 7 258V, and 32 gigabytes of RAM.

TO-DO: Same number of ants, varying number of iterations

4 Results

We report empirical results in four parts: the effect of ant count on solution quality and runtime; the benefit of ACO-seeded branch-and-pivot; a head-to-head comparison of ACO-PN against the θ-heuristic; and an ablation of the prefer-smaller-side (P) and neighbor-scope-limit (N) flags.

4.1 Effect of ant count

We varied ant count n_S across $\{2, 5, 10, 20, 50, 100, 200\}$ using ACO-PN on the full benchmark suite. Figure **??** plots, for each graph, mean percent deviation from the Cui θ-heuristic (among θ-feasible trials), the

θ-feasibility rate, and mean wall-clock time. Solution quality generally improves with more ants but often plateaus before $n_S = 200$; θ-feasibility rises with ant count on most graphs; and runtime scales roughly linearly with n_S on the log-scaled time panel.

4.2 Branch-and-pivot seeding

A marginal increase in the initial lower bound D^* can allow branch-and-pivot to prune more branches and sometimes reduce runtime substantially. ACO was run five times per graph; the largest returned k-DB was used as a seed whenever it beat the θ-heuristic. Branch-and-pivot was then run twice on each eligible graph: once seeded with the θ-heuristic solution and once seeded with the ACO solution.

Table **??** summarizes the 21 graphs on which both pivots completed within a 2000-second timeout (the remaining 15 benchmark graphs timed out under both seeds and are omitted). $|U_G| + |V_G|$ and $|E_G|$ are the vertex and edge counts of the original graph; $|U_R| + |V_R|$ and $|E_R|$ are the corresponding counts after common-neighbor reduction. Rows are grouped into two blocks separated by a horizontal rule: graphs where ACO seeding reduced net end-to-end time (top), and graphs with no net savings or overhead (bottom). Within each block, rows are sorted by ascending absolute time saved.

Time saved is the net wall-time change of using ACO before branch-and-pivot versus running branch-and-pivot on the θ-heuristic seed alone: $T_{\text{pivot}}(\theta) - T_{\text{discovery}} - T_{\text{pivot}}(\text{ACO})$, where $T_{\text{discovery}}$ is the full ACO search cost through the winning replicate (all five runs at the chosen ant count). Positive values mean the ACO-seeded pipeline is faster overall; negative values mean ACO search did not pay for itself (when ACO did not beat the θ-heuristic, the column is $-T_{\text{discovery}}$). *Reduction (%)* is $100 \times$ Time saved$/T_{\text{pivot}}(\theta)$.

4.3 ACO vs. θ-heuristic

We compared ACO-PN (100 ants, five iterations) against the θ-heuristic on the benchmark suite, recording time to completion, iterations to best solution (ITB), and time to best (TTB). On In these trials, ACO took less than 3 seconds longer than the θ-heuristic in the worst case on a graph with 18000 edges, suggesting that it can feasibly be used on much larger graphs. Per-graph results, grouped by outcome, appear in Table **??** (Appendix A).

4.4 Flag ablation (P and N)

We evaluated plain ACO, ACO-P, ACO-N, and ACO-PN on all benchmark graphs at 100 ants. Neighbor scope limit (N) improves solution quality relative to plain ACO and ACO-P; prefer smaller side (P) reduces discovery time on every matched instance, with median discovery time roughly 5× lower than plain ACO at some cost to $|E(D^*)|$.

5 Discussion

5.1 Interpretation of Results

On the tested graphs, ACO outperforms the θ-heuristic on it often finds solutions 3-4x larger – for example, `DNC corecipient`, `moreno`, `nowikinews`, `matter`, and `wikitalknds`. Additionally, in cases where ACO does outperform the θ-heuristic, it results in significantly speed boosts to the brute-force algorithm.

Small improvements in ACO solutions can meaningfully reduce the time that Branch and Pivot takes. On the 6 graphs where ACO beat the θ-heuristic, the net end-to-end pipeline (ACO search plus ACO-seeded pivot) reduced total time by between roughly 50% and 95% relative to a θ-seeded pivot alone.

ACO-N has significantly better solution quality than traditional ACO and ACO-P. We hypothesize this is because, when allowing an ant to add any node possible, in the early stages, an ant takes a subgraph of two nodes that are not connected. By requiring any future additions to share an edge with the most recently visited neighbor, ants are discouraged from creating initial subgraphs with few connections. Indeed, we find that with ACO-N, the average number of missing edges after a subgraph has reached 5 vertices is

5.2 Limitations

Our evaluation of ACO-seeded branch-and-pivot was constrained to 21 small graphs. Branch-and-pivot's runtime grows exponentially with graph size even under a tight bound, and running it to completion on our larger test graphs (e.g., `matter`, `dimacs-polblogs`) was not feasible on the hardware available for this study. As a result, our seeding experiment cannot yet confirm whether the pruning benefit observed on small graphs extends to larger instances. However, it is likely that ACO seeding will scale to larger graphs. We examined all 16 graphs in which ACO tied or lost to the θ-heurstic but only 6 of the 17 graphs in which it won.

Additionally, we injected bicliques with θ vertices on both sides into the graphs. [1] states that their proposed θ-heuristic was inspired by k-MDB with exactly θ vertices on one side and many more vertices on the other side. However, out of the 17 graphs where ACO beat the θ-heuristic, on 8, the ACO solution had exactly θ nodes on one side. This suggests that, even on graphs where the solution meets the condition for which the θ-heuristic was designed, ACO can outperform its solution quality in certain instances.

6 Conclusion

Ant Colony Optimization is a promising heuristic approach to identifying large k-DB. As an emergent field, there do not exist many tested heuristics for this problem. The θ-heuristic, the only previously proposed heuristic we could identify, is significantly faster than our algorithm but often does not produce solutions as large. While ACO takes longer, it can also function as a much more effective upper bound for brute-force search algorithms, cutting net end-to-end search time by as much as 95%.

6.1 Future Work

The graphs excluded from our seeded branch-and-pivot experiment are precisely the ones where we would expect the seeding benefit to be largest. On several large graphs — such as `dblp-cite`, `ciaodvd`, and `matter` — ACO found bicliques roughly 3–4× larger by edge count than the θ-heuristic, at a runtime cost of only a fraction of a second to roughly one second more (Table **??**). Because branch-and-pivot's pruning power depends on the tightness of its initial bound D^*, a substantially larger seed on these graphs should, in principle, prune far more of the search tree than the θ-heuristic's seed does, potentially yielding much larger runtime reductions than we observed on the small graphs in Section 4. We were unable to test this directly because branch-and-pivot's runtime on graphs of this size exceeded the resources available for this study. A natural next step is to rerun the seeded comparison on higher-performance hardware to determine whether this relationship holds at scale.

Additionally, ACO could be adapted to have N subspecies that find N solutions in parallel by using a repulsion mechanism similar to that proposed for diversity in other ant colony variants [9], extending previous multi-colony parallelization strategies [10].

Code is available on `https://github.com/pooch63/aco-kmdb`.

References

[1] D. Cui, R. Li, Q. Dai, H. Qin, and G. Wang, "On the Efficient Discovery of Maximum K-Defective Biclique," arXiv.org. Accessed: Aug. 11, 2026. [Online]. Available: https://arxiv.org/abs/2506.16121

[2] M. Dorigo, V. Maniezzo, and A. Colorni, "Ant System: Optimization by a Colony of Cooperating Agents," IEEE Transactions on Systems, Man and Cybernetics, Part B (Cybernetics), vol. 26, no. 1, pp. 29–41, 1996, doi: 10.1109/3477.484436.

[3] S. Shah, "GCLIQUE: An Open Source Genetic Algorithm for the Maximum Clique Problem," Aug. 2020, doi: 10.36227/techrxiv.12814217.v1.

[4] T. Stützle and H. H. Hoos, "– Ant System," Future Generation Computer Systems, vol. 16, no. 8, pp. 889–914, Jun. 2000, doi: 10.1016/s0167-739x(00)00043-1.

[5] A. E. Rizzoli, R. Montemanni, E. Lucibello, and L. M. Gambardella, "Ant Colony Optimization for Real-World Vehicle Routing Problems," Swarm Intelligence, vol. 1, no. 2, pp. 135–151, Sep. 2007, doi: 10.1007/s11721-007-0005-x.

[6] L. M. Gambardella, É. D Taillard, and M. Dorigo, "Ant Colonies for the Quadratic Assignment Problem," Journal of the Operational Research Society, vol. 50, no. 2, pp. 167–176, Feb. 1999, doi: 10.1057/palgrave.jors.2600676.

[7] K. A. Dowsland and J. M. Thompson, "An improved ant colony optimisation heuristic for graph colouring," Discrete Applied Mathematics, vol. 156, no. 3, pp. 313–324, Feb. 2008, doi: 10.1016/j.dam.2007.03.025.

[8] C. Solnon and S. Fenet, "A Study of ACO Capabilities for Solving the Maximum Clique Problem," Journal of Heuristics, vol. 12, no. 3, pp. 155–180, May 2006, doi: 10.1007/s10732-006-4295-8.

[9] W. Tfaili and P. Siarry, "A New Charged Ant Colony Algorithm for Continuous Dynamic Optimization," Applied Mathematics and Computation, vol. 197, no. 2, pp. 604–613, Apr. 2008, doi: 10.1016/j.amc.2007.08.087.

[10] M. Middendorf, F. Reischle, and H. Schmeck, "Multi Colony Ant Algorithms," Journal of Heuristics, vol. 8, no. 3, pp. 305–320, May 2002, doi: 10.1023/a:1015057701750.

[11] Z. Wang, L. Chang, and J. X. Yu, "Identifying Maximum Defective Bicliques in Large Bipartite Graphs," 2025 IEEE 41st International Conference on Data Engineering (ICDE), pp. 3710–3723, May 2025, doi: 10.1109/icde65448.2025.00277.

[12] H. Yu, A. Paccanaro, V. Trifonov, and M. Gerstein, "Predicting interactions in protein networks by completing defective cliques," Bioinformatics, vol. 22, no. 7, pp. 823–829, Apr. 2006, doi: 10.1093/bioinformatics/btl014.

[13] X. Chen, Y. Zhou, J.-K. Hao, and M. Xiao, "Computing maximum k-defective cliques in massive graphs," Computers & Operations Research, vol. 127, p. 105131, 2021, doi: 10.1016/j.cor.2020.105131.

[14] L. Chang, "Efficient Maximum k-Defective Clique Computation with Improved Time Complexity," Proc. ACM Manag. Data, vol. 1, no. 3, Art. 209, Sep. 2023, doi: 10.1145/3617313.

[15] K. Yu and C. Long, "Maximum k-Biplex Search on Bipartite Graphs: A Symmetric-BK Branching Approach," Proc. ACM Manag. Data, vol. 1, no. 1, Art. 49, Mar. 2023, doi: 10.1145/3588729.

Appendix A ACO vs θ-heuristic

Table **??** lists per-graph results for ACO-PN (100 ants, five iterations) against the θ-heuristic on the full benchmark suite. $|U_G|$, $|V_G|$, and $|E_G|$ are the original graph sizes; $|U_R|$, $|V_R|$, and $|E_R|$ are the reduced graph sizes on which both methods run after common-neighbor reduction.

Rows are grouped into three blocks separated by horizontal rules: graphs where ACO-PN returned a strictly larger θ-feasible $|E(D^*)|$ (or was θ-feasible while the θ-heuristic was not), graphs where the θ-heuristic won under the same rules, and graphs that tied or where neither side achieved θ feasibility on both sides. Within each block, rows are sorted by ACO $|E(D^*)|$.

For ACO, `Discovery` is the cumulative wall time through the winning replicate; `ITB` is the iteration at which that replicate first reached its best $|E(D^*)|$; `TTB` is the elapsed time within that replicate to reach ITB. $|U_{D^*}|$, $|V_{D^*}|$, and $|E(D^*)|$ denote the returned biclique; all times are in milliseconds.

Appendix B Branch and Pivot Pseudocode

Algorithm 4 Branch and Pivot

```
D* ← ThetaHeuristic(G)
function BranchP(S = (U_S, V_S), C = (U_C, V_C))
    if C = ∅ then
        if |E(S)| > |E(D*)| and |U_S| ≥ θ and |V_S| ≥ θ then
            D* ← S
            return
    C_0 ← {v ∈ C | d̄_S(v) = 0}
    if C_0 = ∅ or |C \ C_0| ≥ k − |Ē(S)| then
        u ← argmax_{v∈C} d̄_S
        (S', C') ← Update(S, C, u)
        BranchP(S' ∪ {u}, C')
        BranchP(S, C \ {u})
    else
        u ← argmin_{v∈C_0} d̄_C
        if d̄_C(u) > k − |Ē(S)| > 0 then
            (S', C') ← Update(S, C, u)
            BranchP(S' ∪ {u}, C')
            BranchP(S, C \ {u})
        else
            for v ∈ {u} ∪ N_C(u) do
                (S', C') ← Update(S, C, v)
                BranchP(S' ∪ {v}, C')
                C ← C \ {v}
```

D^* is the k-MDB of the graph.